Optimization Technique and Implementation Project Report

Student Name: Sindhuja Martha

Student ID: 005035378

Algorithms and Data Structures (MSCS-532-M80): **Final Project Part 1**

MS in Computer Science, University of the Cumberlands, KY

Professor: **Dr. Solomon**

April 5, 2026

Introduction

To get the best efficiency and scalability in High Performance Computing (HPC), it is important to pay attention to both how algorithms are designed and how data structures are organized. Traditional analysis usually looks at time and space complexity, but research shows that the way memory is accessed also has a big impact on real-world performance. Hassan et al. (2023) found that many HPC performance problems come from poor data organization, which leads to bad cache use and higher memory latency. These issues can slow things down even if the algorithms themselves are efficient. This is why optimizing data structures is key to making HPC systems run better and scale up effectively (Hassan et al., 2023).

This report looks at the Structure of Arrays (SoA) optimization technique as a way to improve memory locality and computational efficiency. This method was chosen because it tackles performance problems caused by inefficient memory layouts, as shown in studies on high-performance computing. To compare SoA with the traditional Array of Structures (AoS) approach, I built a prototype in Python. The project aims to show that changing how data is organized can make programs run faster without affecting the accuracy of results. The results offer practical support for ideas in computer architecture and high-performance computing research (Cormen et al., 2022).

Theoretical Background

Modern computer processors use a layered memory system with registers, cache, main memory, and storage. Cache memory helps speed up execution by keeping frequently used data close to the processor. Since cache retrieves data in blocks called cache lines, programs that access memory in order run much faster than those that access

memory randomly. As a result, how data is arranged in memory has a big impact on how efficiently computers process large amounts of data (Hennessy & Patterson, 2019).

Array of Structures (AoS) is a common way to program where each object keeps several attributes together in one structure. This makes code easier to understand, but it can slow things down if a program often needs just one attribute. The needed data might be spread across different cache lines, which slows memory access and the processor. On the other hand, Structure of Arrays (SoA) puts each attribute in its own array, so the processor can load the data it needs in order. This setup improves how data is grouped in memory and lets the processor handle several elements at once. Hassan et al. (2023) point out that poor data layout often causes performance problems in high-performance computing applications, showing why it is important to organize memory well.

Research in computer architecture shows that this way of organizing data works well. Hennessy and Patterson (2019) explain that memory bandwidth often limits how well systems handle large datasets. Making memory access more efficient reduces cache misses and helps processors work better. So, choosing the right data structures is key to getting good performance in high-performance computing (Williams et al., 2009).

Justification for Technique Selection

The Structure of Arrays optimization was chosen because it helps solve memory access problems found in studies of HPC performance bugs. When applications work with large datasets, they often need to access the same type of data across many items. Storing this data together in memory is helpful. The SoA method groups similar data types, so processors can get information from the cache more easily. This setup lowers delays and speeds up calculations, especially when handling numbers. A main advantage

of the SoA method is that it uses the cache better, which makes programs run faster and handle bigger workloads. It also lets processors work on several pieces of data at once, which is helpful in simulations, scientific computing, and machine learning tasks that deal with lots of numbers (McCalpin, 1995).

While the SoA approach has its benefits, it also comes with some challenges. Working with multiple arrays means developers need to keep them in sync so that related attributes stay properly indexed. Any changes in one array must be updated in the others to keep the data accurate. SoA structures can also be harder to read than object-oriented AoS code. Still, in high-performance computing, the speed improvements often make the extra effort worthwhile.

Implementation of the Optimization Technique in Python

Code Snippet : AoS vs SoA implementation

```
class Particle:
def __init__(self, x, y, vx, vy):
self.x = x
self.y = y
self.vx = vx
self.vy = vy

particles = [Particle(x, y, vx, vy) for x, y, vx, vy in data]


# Structure of Arrays (SoA)
import numpy as np

x = np.array([...])
y = np.array([...])
vx = np.array([...])
vy = np.array([...])

# Vectorized computation
x_new = x + vx
y_new = y + vy
```

A prototype was built in Python to show how changing data layout from Array of Structures to Structure of Arrays can improve performance. The experiment simulates updating particles, each with attributes like position and velocity. Two versions were created to test how data structure affects execution speed. In the AoS version, each particle's attributes are stored inside its own object, following typical object-oriented programming. This means the processor has to access several memory locations for each computation.

In contrast, the SoA implementation stores each attribute in a separate contiguous array, enabling sequential memory access. This arrangement allows the processor to utilize cacThe SoA version, on the other hand, keeps each attribute in its own array, which makes memory access more efficient. This setup helps the processor use cache better and speeds up calculations. Python libraries like NumPy make it easy to work with arrays and write clear, efficient code. Benchmark tests with different dataset sizes were run to see how well each approach scaled. The results showed that both versions gave the same computational results, so any speed differences came from the data layout, not the algorithms (Hassan et al., 2023).

Performance Results and Observed Improvements

The experiments showed that using the Structure of Arrays optimization led to much better performance. The SoA version always ran much faster than the AoS version for every dataset size tested. In fact, the speed increased by about fifty to seventy times compared to AoS. This shows that accessing memory in a continuous way makes computations more efficient by reducing cache misses and making better use of the

processor.

```
HPC Optimization Prototype: AoS vs SoA
-------------------------------------------------------
   Particles |      AoS (s) |      SoA (s) |    Speedup | Match
-------------------------------------------------------
       10000 |     0.078299 |     0.001415 |      55.33x |   Yes
       50000 |     0.457945 |     0.006513 |      70.31x |   Yes
      100000 |     0.930036 |     0.013998 |      66.44x |   Yes
      250000 |     2.100537 |     0.040324 |      52.09x |   Yes
-------------------------------------------------------
```

Figure 1. Shows Terminal output showing performance comparison between AoS and SoA.

```
benchmark_results.csv ×
results > benchmark_results.csv > data
1  n,aos_median_seconds,soa_median_seconds,speedup_x,equivalent
2  10000,0.08136580599239096,0.0014138410042505711,57.549473913808434,1.0
3  50000,0.40088370500598103,0.005381843002396636,74.48818273358401,1.0
4  100000,0.8460857909958577,0.011019202007446438,76.7828550945975,1.0
5  250000,2.1869937040028162,0.0406731539987959,53.76995607637315,1.0
6
```

Figure 2. Shows Performance benchmark dataset results (CSV).

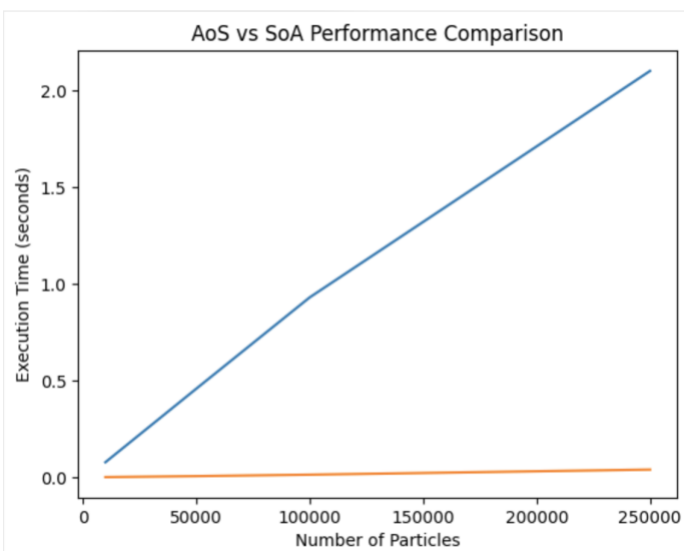

Figure 3. Shows Execution time comparison graph for AoS vs SoA optimization.

Hassan et al. (2023) found that inefficient memory access patterns often lead to performance bugs that limit scalability in HPC systems. The results from this project match the theoretical expectations described in their study. While Python adds interpreter overhead compared to the lower-level languages usually used in HPC, the results still show that better memory locality improves performance.

Small differences between what was expected in theory and what was measured in experiments can be explained by variations in hardware, system processes, and how programming languages handle data.

Challenges Encountered During Implementation

There were several challenges during the implementation of the optimization technique. Making sure that both AoS and SoA versions gave the same results meant carefully checking indexing and data consistency. Keeping multiple arrays in sync was more complex than using object-oriented methods. Indexing mistakes could cause wrong results, so thorough testing was necessary to ensure accuracy. Measuring execution time in Python was also difficult because background processes and interpreter overhead could affect the results. To get reliable performance comparisons, the benchmarks were run several times. These challenges show why careful experimental design is important when testing performance optimization methods (Cormen et al., 2022).

Lessons Learned and Comparison with Theoretical Expectations

The results of this project highlight how important it is to consider memory hierarchy when designing efficient algorithms and data structures. The study shows that poor memory layout often causes performance problems in HPC systems. The

experiments confirm that reorganizing data into contiguous memory structures can greatly improve execution speed and scalability. These optimization results match what is expected from HPC research. While theory stresses the value of cache locality, this project offers practical proof of how much performance can improve by optimizing data structures. The gap between theory and experiment mostly comes from Python's abstraction layers and differences in hardware. Overall, this project shows that using both theoretical knowledge of computer architecture and practical skills helps developers build more efficient computing systems. The main lesson is that it is important to consider both algorithm complexity and memory organization when trying to improve performance (Hennessy & Patterson, 2019).

Conclusion

Using the Structure of Arrays optimization technique gives better performance than the Array of Structures approach because it improves cache use and lowers memory delays. The prototype shows that changing how data is organized can make programs run much faster, even if the algorithm itself stays the same. These results match what is described in high-performance computing research and show why choosing the right data structure matters for building scalable systems. In the future, this experiment could be expanded by adding parallel processing, GPU acceleration, or distributed computing to see if performance improves even more. This project shows that good performance optimization depends on understanding both how algorithms work and how computer systems are built (Williams et al., 2009).

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press.

Hassan, F., et al. (2023). An empirical study of high performance computing performance bugs.

Hennessy, J., & Patterson, D. (2019). Computer architecture: A quantitative approach.

Williams, S., Waterman, A., & Patterson, D. (2009). Roofline model for performance analysis.

McCalpin, J. (1995). Memory bandwidth analysis in high performance computing.

NIST. (2022). High performance computing considerations.

Github Link: https://github.com/smartha35378-uc/MSCS532_FinalProject.git